CORAL GARDENERS

UC San Diego

Team Coral Gardeners

University of California San Diego

Place-Specific Reef Fish Identification for Coral Restoration

Project Plan

Spring 2026

Project Sponsor

Richard Williams

Team Members

Vibusha Vadivel

Marlyn Arque Rupa

Aleksa Popovic

Ewan Shen

Table of Contents

Executive Summary

Healthy coral reefs depend not only on coral cover, but also on the fish communities that live around them. Fish abundance and diversity are an important dimension of reef health, yet monitoring fish biodiversity in tropical coral reef ecosystems is difficult and time-consuming. Recent advances in computer vision make analyzing reef imagery and automating fish census tasks possible.

This project tackles that question directly by building a computer vision pipeline for place-specific reef fish identification. The goal is to support coral restoration efforts by making reef monitoring faster, more scalable, and more automated.

Project Charter

Our project focuses on building a computer vision pipeline for identifying fish in reef imagery. In the first step, the system will detect fish within an image taken underwater. In the second step, each detected fish will be passed to a classifier that predicts its species using a model tailored to a specific geographic region.

The regional aspect is an important part of the project. Fish species vary across reef locations, and we plan to use publicly available labeled datasets, such as iNaturalist and GBIF, to build more location-specific classifiers using available geolocation metadata. Our classifier pipeline will also account for camouflage, camera lighting, and fish age.

Our goal is to package the classifier pipeline into a Python package that can take an input image, identify fish present in the scene, and return classification results. The final deliverable will also include documentation as well as meaningful unit and functional tests so that the system is usable, understandable, and reproducible.

Project Approach

Fish detection or segmentation
The presentation suggests a first step based on finding fish in the image, possibly using something like Segment Anything. The team could compare segmentation-based and detection-based approaches for isolating fish from cluttered reef backgrounds.

Location-specific classification
Once fish are isolated, train a classifier to recognize fish relevant to a specific region. A fine-tuned YOLO classifier is suggested in the presentation as one possible direction.

Public-data training pipeline

Use public sources such as iNaturalist and GBIF to assemble training data and use location metadata to bias the classifier toward species likely to appear at a given site.

Evaluation on reef imagery

The slides report about **1,800 images from a site in Moorea, French Polynesia**, with fish marked by bounding boxes and labeled by functional group. These data could be used both to develop a segmented fish dataset and to test classifier performance.

Minimum Viable Product

- **A working Python package that can detect or segment fish in reef images**

  The core deliverable is an installable Python package implementing a two-stage pipeline. A Segmenting Anything model followed by a fine-tuned YOLO classifier for identification. The goal of the models is to run the full detection and classification pipeline on a given image and return the type of fish.

- **A classifier that can be trained for a specific location**

  Rather than a static model, the model is trainable for a specific location. Given a reef location (GPS coordinates or a named site), the classifier should be able to pull labeled fish observations from iNaturalist and GBIF filtered to that region, generate training fish crops, and fine-tune the YOLO classifier on the resulting dataset. For the MVP, the target site is Moorea, French Polynesia, which already has 1,800 labeled images available for training and evaluation. This data would be used for training and evaluation for our YOLO classifier.

- **Initial evaluation on the Moorea dataset or similar reef imagery**

  The MVP will be validated against the Moorea labeled dataset as a concrete benchmark. Running the Moorea dataset early serves two purposes:

  1) It confirms that the pipeline works on real representative images.
  2) Establishes a baseline accuracy number.

  If the Moorea dataset is insufficient, we can find similar publicly available reef imagery as a fallback to ensure the evaluation is realistic in underwater conditions.

- **Documentation and meaningful unit and functional tests**

  The MVP will have proper documentation and a test suite sufficient for another developer to install, run, and extend the package. Unit tests will cover the data pipeline, the SAM (Segmenting Anything Model) segmentation state, and the YOLO

inference. Functional tests will validate the full end-to-end pipeline on a sample Moorea image, asserting that the output format is correct and that detections are produced without errors. Documentation will include a README with installation steps, core functions, and notes on the model's training location.

Stretch Goals (Post MVP)

Improved Segmentation for Challenging Visual Cases

- Segmentation performance may degrade in scenarios such as overlapping fish, partial occlusion, or cluttered reef backgrounds where boundaries are unclear. As a stretch goal, we aim to improve segmentation in these cases by experimenting with more advanced models, post-processing techniques and hybrid approaches that combine detection with coarse segmentation or refined bounding boxes.

Functional-Group Prediction in Addition to Species Classification

- Species-level classification may be too fine-grained for consistent performance and so a stretch goal, we will extend the system to support functional-group prediction (e.g., herbivores, carnivores) as either a complement or alternative to species labels.

Detection of Non-Native or Invasive Species

- Because the model may be trained on region-specific data, it may struggle to correctly identify fish that are not native to a given location. As a stretch goal, we investigate methods for handling out-of-distribution detection, such as confidence-based filtering or anomaly detection techniques. If reliable classification is not possible, the system may instead flag uncertain or unfamiliar observations rather than forcing an incorrect prediction.

Adaptable Pipeline for New Reef Locations

- If time permits, we want to develop a workflow that can be efficiently adapted to new reef environments with minimal retraining. This means being more meticulous in modularizing the workflow, documenting retraining steps, and leveraging transfer learning or domain adaptation techniques.

Risk Analysis

Underwater Image Quality and Fish Camouflage

- **Difficulty:** High
- **Risk:** Reef images are much harder to work with than standard fish photographs. Fish may be small, partially hidden, blurred by motion, affected by poor lighting, or visually blended into the coral background. Some fish are naturally camouflaged and can be difficult to separate from reef structures even for human observers. This makes fish detection or segmentation one of the most challenging parts of the project and could reduce the performance of the full pipeline.
- **Mitigation:** We will reduce this risk by testing detection methods early on a subset of reef data and comparing simpler detection-based approaches with segmentation-based ones. We can also use preprocessing and augmentation techniques to improve robustness to underwater conditions. If camouflage remains a major obstacle in still images, we may explore whether sequential frames or motion cues from video could help make fish easier to identify.

Domain Gap Between Public Data and Reef Images

- **Difficulty:** High
- **Risk:** Publicly available fish datasets such as iNaturalist or GBIF are likely to contain well-lit, high-resolution images of individual fish, while the reef images in this project are naturally lit, cluttered, and often lower resolution. A model trained only on public images may not generalize well to the actual reef setting.
- **Mitigation:** We will address this by using the Moorea reef data as an evaluation benchmark and, when possible, as part of the development loop. We can also experiment with degrading public images or applying augmentations so that the training data better resembles real reef conditions. If needed, we can reduce the scope of the classifier to a smaller number of common species or functional groups for the MVP.

Limited Time and Project Scope

- **Difficulty:** Medium
- **Risk:** The quarter is short, and this project includes multiple major components: fish detection, fish classification, public-data collection, geolocation-aware filtering, packaging, documentation, and testing. Attempting to fully optimize every component could lead to an incomplete final system.
- **Mitigation:** We will define a clear minimum viable product early. The MVP will focus on a working end-to-end pipeline for still images from one location, likely Moorea, with a manageable subset of species or labels. Stretch goals such as video support,

broader geographic adaptation, and stronger geolocation-aware modeling will only be pursued if the core pipeline is stable.

Model Performance May Be Lower Than Expected

- **Difficulty:** Medium
- **Risk:** Because this project combines difficult detection and fine-grained classification in a challenging visual environment, model accuracy may fall below expectations, especially rare and camouflaging species or difficult images.
- **Mitigation:** We will set realistic evaluation goals and prioritize demonstrating a functioning and well-structured pipeline over claiming perfect performance. If species-level classification is too difficult, we can narrow the task to a subset of species or functional-group prediction while still delivering a meaningful result.

Group Management

Our group's major roles are based on the main technical parts of the project, although they are not completely fixed yet since we are still early in the planning process. In general, all of us will contribute as both software developers and researchers, but each person will likely take more responsibility in certain areas. Aleksa will likely take on more of a project management and machine learning engineering role, helping keep the team organized while also contributing to model development. Vibusha will likely focus more on data engineering tasks such as dataset preparation, preprocessing, and training pipeline support. Marlyn will likely work mainly on the machine learning side through model development and experimentation. Ewan will likely take more responsibility for the computer vision side, especially detection and segmentation work.

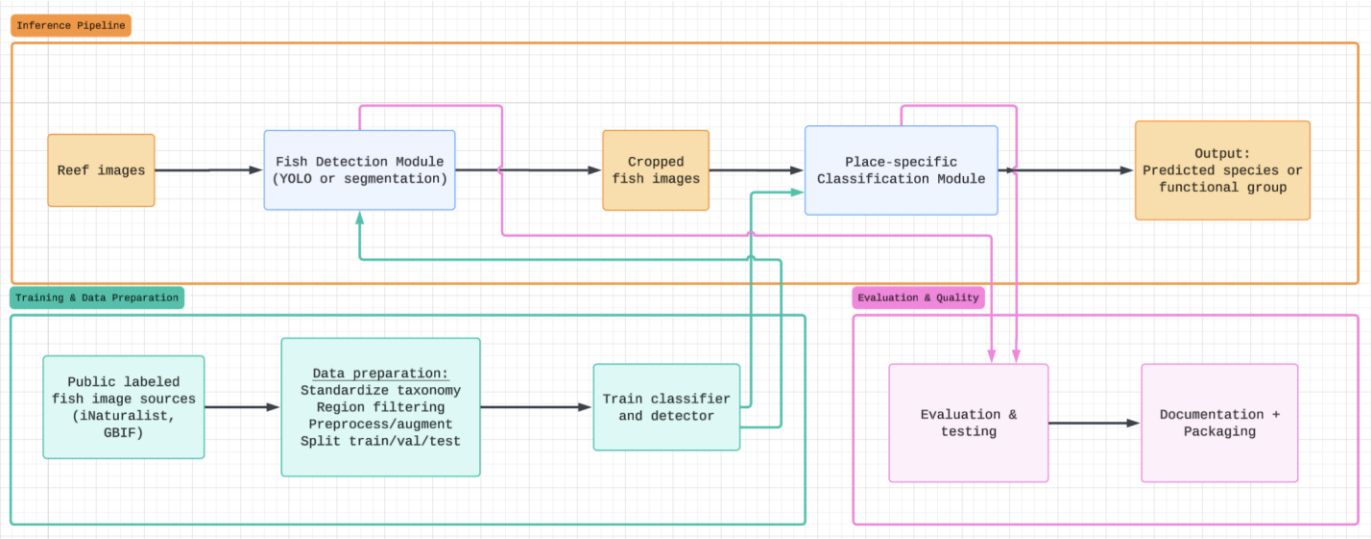
Our decisions will be made by consensus rather than by one team leader. Since the project involves multiple connected components, we want technical and planning decisions to be discussed as a group so that everyone stays aligned and has input on the direction of the project.

For communication, we are currently using a text-message group chat and a Discord server. The group chat is useful for quick updates, while Discord gives us a more organized space for technical discussion and sharing materials. We have also scheduled weekly meetings with our project sponsor, Rich Williams, every Friday at 11:00 a.m., which will help us stay on track and get regular feedback.

To manage schedule and deliverables, we plan to use sprints along with a Gantt chart to track tasks, deadlines, progress, and ownership. This will help us see early if we are falling

behind. If schedule slips happen, we will address them through sprint check-ins, reassign tasks when needed, and narrow scope if necessary so that we still deliver a strong core system. The same planning process will also make it clear who is responsible for each deliverable and milestone throughout the quarter.

Project Development



System Architecture

The system follows a two-stage computer vision pipeline for place-specific reef fish identification. First, reef images or sampled video frames are passed into a fish detection module, which identifies the locations of fish within the full underwater scene. This step can be implemented using a lightweight object detector such as YOLO, or a segmentation-based model if more precise fish isolation is needed. The output of this stage is a set of cropped fish images or regions of interest extracted from the original reef image.

Next, each cropped fish image is passed into a place-specific classification module. This classifier is trained using publicly available labeled fish images, such as data collected from iNaturalist and GBIF, filtered by geographic region so that the model focuses on species likely to appear at the target reef location. The classifier then predicts the most likely species, or functional group if species-level classification is too difficult, for each detected fish.

To support this classification stage, the system also includes a data preparation and training pipeline. This component gathers public fish images, standardizes taxonomy labels, filters data by region, applies preprocessing or augmentation, and prepares datasets for training and evaluation. The trained detection and classification models are

then integrated into a Python package that provides a simple interface for users to input reef images and receive fish predictions as output.

Finally, the full system includes an evaluation and testing layer to measure detection and classification performance on reef imagery such as the Moorea dataset, along with documentation and unit/functional tests to ensure the package is reliable and reusable.

Development Roles

Role	Responsibilities
Detection	Build and tune the Stage 1 detector, compare YOLO vs. segmentation approaches, handle cropping logic
Classification	Train and evaluate the Stage 2 classifier, experiment with geolocation-aware training
Data Pipeline	Pull from iNaturalist and GBIF, clean and standardize taxonomy, manage train/val/test splits
Packaging & QA	Wrap the pipeline as a pip-installable Python package, write unit and functional tests, maintain README, set up CI and the evaluation harness

Hardware and Software

Language and editor: Python 3.13 in VSCode with Git.

Core ML libraries: PyTorch (≥2.0) for training, Ultralytics YOLOv8 for Stage 1 detection, Segment Anything (SAM) as a segmentation fallback for occluded fish, torchvision and albumentations for preprocessing and augmentation, and scikit-learn for evaluation metrics and stratified splits.

Data Sources: iNaturalist API for labeled fish observations, GBIF API for region-filtered species lists, and the provided Moorea reef dataset

Packaging and Development tools: pyproject.toml for pip-installable packaging, GitHub for source hosting, GitHub Actions for CI, pytest for tests, ruff for linting, pre-commit hooks, Sphinx or MkDocs for API docs, and Jupyter for tutorial notebooks.

In terms of hardware, this project will require access to a GPU for training and testing computer vision models efficiently, especially since we will be segmenting and fine-tuning on reef imagery. We will also need sufficient local or cloud storage for reef images, public datasets, and any intermediate processed outputs. On top of the software dependencies

mentioned above, we may depend on an image labeling tool for segmentations and annotations (to serve as context during the model training).

Testing

Unit tests cover individual functions, data loading, taxonomy normalization, geographic filtering, image preprocessing, the cropping step between Stage 1 and Stage 2. These will run on every commit via GitHub Actions and will be written using pytest. Target coverage is ~80% for non-ML utility code.

Functional tests will cover the full pipeline end-to-end on a small fixture set of reef images. We will be loading a reef image that goes through the public API. Then the image will be passed to the YOLO detector which will return a list of bounding boxes to classify the image. Each bounding box is cropped from the original image which will be passed to the classifier along with the location metadata. Finally, the prediction containing the species, functions group label, a confidence score or a functional-group fallback. The final object returned by the public API is validated against the documented schema (a list of dicts with keys box, crop_shape, species, functional_group, species_confidence, and group_confidence). Missing keys, wrong types, or malformed bounding boxes all fail the test. The same fixture image is run through the pipeline twice and the outputs are compared to ensure determinism.

Model evaluation will measure actual performance on held-out reef imagery. For detection, we will track precision, recall, and mean Average Precision against bounding-box labels. For classification, we will track top 1 and top 5 accuracy, and a confusion matrix at the functional-group level. The Moorea dataset is our primary evaluation set. So, we will hold out a fixed portion of it from the start of the project and never train on it so our reported numbers are honest.

Documentation

- **Package Documentation,** which is pip-installable. It exposes a top-level identify() function for end-to-end inference, plus helpers for training a classifier on a new location and running detection only
- **README** with installation, a quick start example, and a description of the two-stage pipeline
- **Docstrings** on every public function and class, using a consistent format (Google or NumPy style) so they can be auto-rendered
- **Tutorial notebook** walking through the full flow on how to building a classifier for a new location, running inference on a reef image, interpreting outputs

- **Developer guide** explaining how to pull fresh iNaturalist/GBIF data, retrain for a new region, and run the test suite

Project Milestones and Schedule

Milestone Overview

Required:

- Baseline pipeline and dataset (Week 4)
- Robustness improvements (camouflage, lighting, age) (Weeks 5–6)
- MVP system and evaluation (Weeks 7–8)
- Final system and report (Weeks 9–10)

Important:

- CLI-based inference system
- Performance optimization and usability improvements

Stretch Goals:

- Improved segmentation for challenging cases (overlapping or partially visible fish)
- Integration with broader reef monitoring and restoration systems
- Real-time video inference

Milestone Deliverables Overview:

- **Week 4:** Baseline model demonstrated through sample predictions and evaluation graphs
- **Week 6:** Performance improvements demonstrated through before vs after comparison charts on challenging conditions
- **Week 8:** Working system demonstrated via CLI demo and documented GitHub repository
- **Week 10:** Final system demonstrated through presentation, report, and complete integrated pipeline

Week 1 (3/30 - 4/3)

- Project familiarization
- Class Overview

Week 2 (4/6 - 4/10)

- Initial meeting with team members
- Initial meeting with project sponsors
- Understand project requirements & project approach

Week 3 (4/13 - 4/17)

Tasks

- Start sprint 1
- Research Segment Anything models
- Submit project specification

Week 4 (4/20 - 4/24)

Tasks

- **Alex:** Build initial pipeline skeleton (data → training → evaluation → inference)
- **Ewan:** Train baseline YOLO model and verify detection outputs
- **Marlyn:** Implement evaluation scripts and compute baseline metrics
- **Vibusha:** Clean and format dataset into YOLO-compatible structure

Deliverables

- **Alex:** Working pipeline skeleton (end-to-end execution)
- **Ewan:** Baseline trained YOLO model with sample predictions
- **Marlyn:** Baseline evaluation report (metrics + graphs)
- **Vibusha:** Cleaned and formatted dataset

Week 5 (4/27 - 5/1)

Tasks

- Start sprint 2
- **Alex:** Design evaluation subsets and analysis plan for challenging conditions (camouflage, lighting, age variation)
- **Ewan:** Identify and categorize failure cases from baseline predictions
- **Marlyn:** Analyze performance degradation across failure cases and quantify baseline weaknesses
- **Vibusha:** Create augmented dataset targeting identified challenges (lighting shifts, occlusion, variation)

Week 6 (5/4 - 5/8)

Tasks

- **Alex:** Integrate improved model into pipeline and validate performance gains on challenging subsets
- **Ewan:** Implement preprocessing techniques (contrast enhancement, normalization, augmentation-aware input)
- **Marlyn:** Retrain model and evaluate improvements against baseline and failure subsets
- **Vibusha:** Finalize augmented dataset and ensure reproducible data pipeline

Deliverables

- **Alex:** Validated system showing improved robustness on challenging conditions
- **Ewan:** Preprocessing pipeline integrated into inference workflow
- **Marlyn:** Performance comparison report (before vs after on difficult cases)
- **Vibusha:** Augmented dataset + reproducible preprocessing pipeline

Week 7 (5/11 - 5/15)

Tasks

- Start sprint 3
- **Alex:** Prepare and validate full system demo pipeline including robustness improvements
- **Ewan:** Debug and stabilize CV pipeline for consistent inference
- **Marlyn:** Evaluate model on unseen data and confirm generalization
- **Vibusha:** Validate dataset performance across varied environmental conditions

Week 8 (5/18 - 5/22)

Tasks

- **Alex:** Finalize evaluation pipeline and optimize system integration for usability
- **Ewan:** Optimize inference speed and clean CV pipeline code
- **Marlyn:** Generate final evaluation metrics and visualizations across all test cases
- **Vibusha:** Document dataset pipeline and ensure reproducibility

Deliverables

- **Alex:** Final integrated system with evaluation pipeline
- **Ewan:** Optimized and clean CV codebase
- **Marlyn:** Final performance metrics and evaluation graphs

- **Vibusha:** Fully documented dataset pipeline

Week 9 (5/25 - 5/29)

Tasks

- **Alex:** Validate system performance on edge cases and finalize presentation workflow
- **Ewan:** Finalize inference pipeline and demo system
- **Marlyn:** Conduct edge case testing and error analysis
- **Vibusha:** Perform final dataset validation and consistency checks

Week 10 (6/1 - 6/5)

Tasks

- **Alex:** Compile final report and ensure full system integration for submission
- **Ewan:** Write CV methodology section and finalize demo outputs
- **Marlyn:** Write evaluation/results section with robustness analysis
- **Vibusha:** Write dataset/preprocessing section

Deliverables

- **Alex:** Final report + complete integrated system
- **Ewan:** CV pipeline documentation + demo output
- **Marlyn:** Evaluation section including robustness results
- **Vibusha:** Dataset documentation + preprocessing writeup